

# Lecture 4 — Inheritance in Python

- Goal: Understand inheritance step by step
- Focus: Base & derived classes, constructors, `super()`, overriding
- No polymorphism in this lecture

# Why We Need Inheritance?

- Avoids code duplication
- Allows reuse of existing logic
- Models real-world 'is-a' relationships
- Example: Car is a Vehicle, Dog is an Animal

# What is Inheritance?

- Inheritance = creating a new class (child) from an existing one (parent)
- Child gets attributes & methods of the parent automatically
- We can extend or override behavior in the child

# Inheritance Syntax

```
class Parent:  
    pass  
  
class Child(Parent):  
    pass
```

# Base Class Example

- Animal is the base class
- It has an `__init__` method (constructor)
- It has a `speak()` method

```
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        print(self.name, "makes a sound")
```

# Child Class Example

- Dog inherits from Animal
- No new code yet, but Dog gets Animal's methods
- Output: Buddy makes a sound

```
class Dog(Animal):    # Dog inherits from Animal
    pass
```

```
d = Dog("Buddy")
d.speak()
```

# Adding Extra Features in Child

- Child classes can add new methods
- Dog now has both speak() (inherited) and bark() (new)

```
class Dog(Animal):  
    def bark(self):  
        print(self.name, "barks loudly")  
  
d = Dog("Buddy")  
d.bark()
```

# Constructors and Inheritance

- If child has no constructor, parent's constructor is used
- If child defines its own, parent's constructor is not called automatically
- We use `super()` to call the parent constructor



# Using super()

- `super().__init__(name)` initializes parent part
- Child can add its own attributes

```
class Cat(Animal):
    def __init__(self, name, color):
        super().__init__(name)    # call Animal constructor
        self.color = color

c = Cat("Milo", "black")
print(c.name, c.color)
```

# Method Overriding Explained

- Child class can redefine a method with same name as parent
- New definition replaces the old one for child objects
- Parent method is still available via `super()` if needed

# Dog Overrides speak()

- Child Dog replaces parent's speak()
- Output: Buddy says Woof!

```
class Dog(Animal):  
    def speak(self):    # overrides Animal's speak  
        print(self.name, "says Woof!")  
  
d = Dog("Buddy")  
d.speak()
```

# Calling Parent Method with super()

- Child method calls both parent and its own logic
- Output: Buddy makes a sound Buddy says Woof!

```
class Dog(Animal):  
    def speak(self):  
        super().speak()    # call parent version  
        print(self.name, "says Woof!")  
  
d = Dog("Buddy")  
d.speak()
```

# Example: Banking Accounts

- Account is the base class
- BusinessAccount overrides withdraw to allow overdraft

```
class Account:
    def __init__(self, owner, balance=0.0):
        self.owner = owner
        self.balance = balance

    def withdraw(self, amount):
        if amount > self.balance:
            return "Insufficient funds"
        self.balance -= amount
        return self.balance

class BusinessAccount(Account):
    def __init__(self, owner, balance=0.0, overdraft=5000):
        super().__init__(owner, balance)
        self.overdraft = overdraft

    def withdraw(self, amount): # override
        if amount > self.balance + self.overdraft:
            return "Exceeds overdraft limit"
        self.balance -= amount
        return self.balance
```

# Best Practices for Inheritance

- Use inheritance only for true 'is-a' relationships
- Keep base classes small and focused
- Use `super()` when overriding constructors
- Document overridden behavior clearly
- Prefer composition if 'has-a' fits better

# Problem 1 — Vehicle System

- Design Vehicle base class with brand, model
- Create Car subclass (adds num\_doors)
- Create Bike subclass (adds type: mountain/road)
- Override display() in each subclass to show details

## Problem 2 — Employee System

- Employee base class with name, base salary
- Developer subclass adds hourly pay
- Manager subclass adds bonus
- Override `monthly_pay()` accordingly



## Problem 3 — Banking System

- Account base class with deposit, withdraw
- SavingsAccount adds add\_interest()
- BusinessAccount overrides withdraw for overdraft
- Test behavior of all accounts